

Set and get

The basic operation in Redis is to set values for a key. This can be done with the following command:

```
set magazine:name "open source for you"
```

In this example, *magazine:name* is the key. The value stored at the key is *open source for you*.

Once a key is set, its value can be retrieved any time with the help of the GET command:

```
get magazine:name
```

This command will fetch the value of the key *magazine:name*, i.e., *open source for you*.

Checking whether a key exists

The command *exists* is used to see whether a key exists or not. If the key is present, it returns the value 1; otherwise, the value 0 is returned:

```
exists magazine:name
(integer) 1
```

```
exists magazine:month
(integer) 0
```

Increment and decrement values

The numeric values stored in keys are incremented or decremented using *incr* and *decr*, respectively:

```
set age 25
OK
incr age
(integer) 26
```

In this example, a key *age* is set to 25 and the *incr* command has incremented its value by one, i.e., to 26. Similarly, the values can be decremented with the *decr* command:

```
> set month 10
OK
```

```
> decr month
(integer) 9
```

Increment and decrement by a specific offset can be done with the *incrby* and *decrby* commands:

```
> incrby age 5
(integer) 31
> decrby month 2
(integer) 7
```

Setting the expiry for a key

Redis provides a method to set the expiry duration for a key. This can be done with the *expire* command:

```
set seatcount 10
OK
> expire seatcount 180
(integer) 1
```

In this example, a key *seatcount* is set to the value 10 and the *expire* command is used to set an expiry of 180 seconds. At any point, the *ttl* command can be used to check the time till when the key will be available. An example is as follows:

```
> expire seatcount 180
(integer) 1
> ttl seatcount
(integer) 170
> ttl seatcount
(integer) 109
> ttl seatcount
(integer) 29
> ttl seatcount
(integer) -2
```

The *ttl* command shows the time that remains for the key to expire, which reduces as time passes. Once the time limit is reached, negative values are returned.

List data structure

As stated earlier, Redis has various complex data structures. In this section, the operations related to a List data structure are given. A List is a series of

ordered values. Some of the important commands associated with it are:

- *rpush*
- *lpush*
- *llen*
- *lrange*
- *lpop*
- *rpop*

rpush is used to add a new element at the end of the List and *lpush* adds an element at the beginning of a List. The code snippet is shown below:

```
> rpush months "Jan"
(integer) 1
> rpush months "Feb"
(integer) 2
> rpush months "Mar"
(integer) 3
```

The *llen* command is used to return the number of items in the List:

```
> llen months
(integer) 3
```

The command *Lrange* is used to display the elements of the List:

```
> Lrange months 0 -1
1) "Jan"
2) "Feb"
3) "Mar"
```

lpush is used to add the values at the beginning of the List, as shown below:

```
> lpush months "Dec"
(integer) 4
> Lrange months 0 -1
1) "Dec"
2) "Jan"
3) "Feb"
4) "Mar"
```

Similarly, items can be removed from the beginning and the end of the List with *lpop* and *rplop* commands, as shown below:

```
> rpop months
"Mar"
```